O'REILLY®

Harness Engineering for AI Agents

Meet your Instructor

Richmond Alake

- **Director of AI Developer Experience at Oracle:** Leads AI developer experience and outreach for Oracle AI Database, focusing on open-source framework integrations, AI ecosystem, and developer enablement.
- **AI Engineer & Computer Vision Specialist:** Holds an MSc in Computer Vision, Machine Learning and Robotics from the University of Surrey, with software development and ML engineering experience since around 2016. Former ML Architect at Slalom Build and AI/ML Developer Advocate at MongoDB.
- **Technical Writer & Educator**: Authored 200+ technical articles with over 1 million views across Medium, MongoDB Developer Hub, and the NVIDIA Developer Blog. Co-instructor of a DeepLearning.AI short course alongside Andrew Ng's team and lead instructor for an O'Reilly live training bootcamp on Agent Memory.
- **Memory Engineering Pioneer**: Coined "Memory Engineering" as a named AI discipline and built a sustained public identity around it through the #100DaysOfAgentMemory series.
- **AI Stack Engineer**: Creator of MemoRizz (open-source agent memory library) and the Oracle AI Agent Memory Package.

Content Overview

1. **Defining the AI agent harness**: a first-principles deconstruction of AI, agent, and harness, the agent loop, and the assembled definition, agent = model + harness.

2. **Why a harness is required**: why model capability alone doesn't ship, why scaling doesn't guarantee reliable outcomes, and how the harness turns decisions into actions.

3. **Components of an effective harness**: the six runtime components (observation interface, context manager, action interface, state and artifacts, verify and guardrails, control loop) and the long-horizon mechanisms layered on top: planning, memory, tool use, feedback control.

4. **The agentic spectrum**: from chat interfaces, RAG agents, and agentic RAG through LLM workflows to autonomous agents, moving from human-directed fixed paths to model-directed open goals.

5. **Application mode thinking**: the three modes an agent ships in, assistant (user-in-the-loop), workflow (predefined sequence), and deep research (autonomous), and how the mode determines harness design.

6. **The memory-first agent harness**: memory as the substrate rather than a bolted-on feature, the retain, recall, reuse, refine cycle, and why memory allocation precedes every other harness design decision.

7. **Assistant mode, memory-first**: implementing working, episodic, semantic, and procedural memory for the user-in-the-loop harness, integrated with tools, Skills, MCP servers, a sandbox, and a semantic cache at the harness boundary.

8. **Shipping the harness to production**: the reference architecture, a Next.js frontend on Vercel talking to a stateful backend beside Oracle AI Database on OCI, with LangSmith streaming traces and evals from the harness.

PART 01

Defining AI Agent Harness

AI · agent · harness · the agent loop

[AI] [Agent] [Harness]

Defining the term, word by word [AI]

01 / [AI]

The raw capability

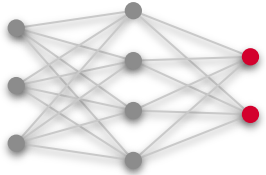
A computational form of intelligence — today, effectively the large language model. Transformer architectures, millions to trillions of parameters, “world compressors” that learn reasoning and language from vast amount of data.

The evolution of neural networks

01

Feedforward network

1986 · BACKPROP



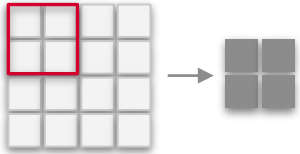
Dense layers map a fixed input to an output. Every unit connects to every unit in the next layer.

no structure · no sequence

02

Convolutional network

1998 · LENET



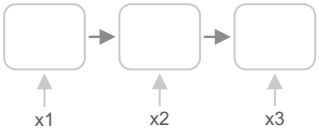
Shared filters slide across the input, finding local patterns that build into visual features.

sees locally, pools globally

03

Recurrent network

1997 · LSTM



Processes a sequence step by step, carrying hidden state forward from token to token.

forgets long range · hard to parallelise

04

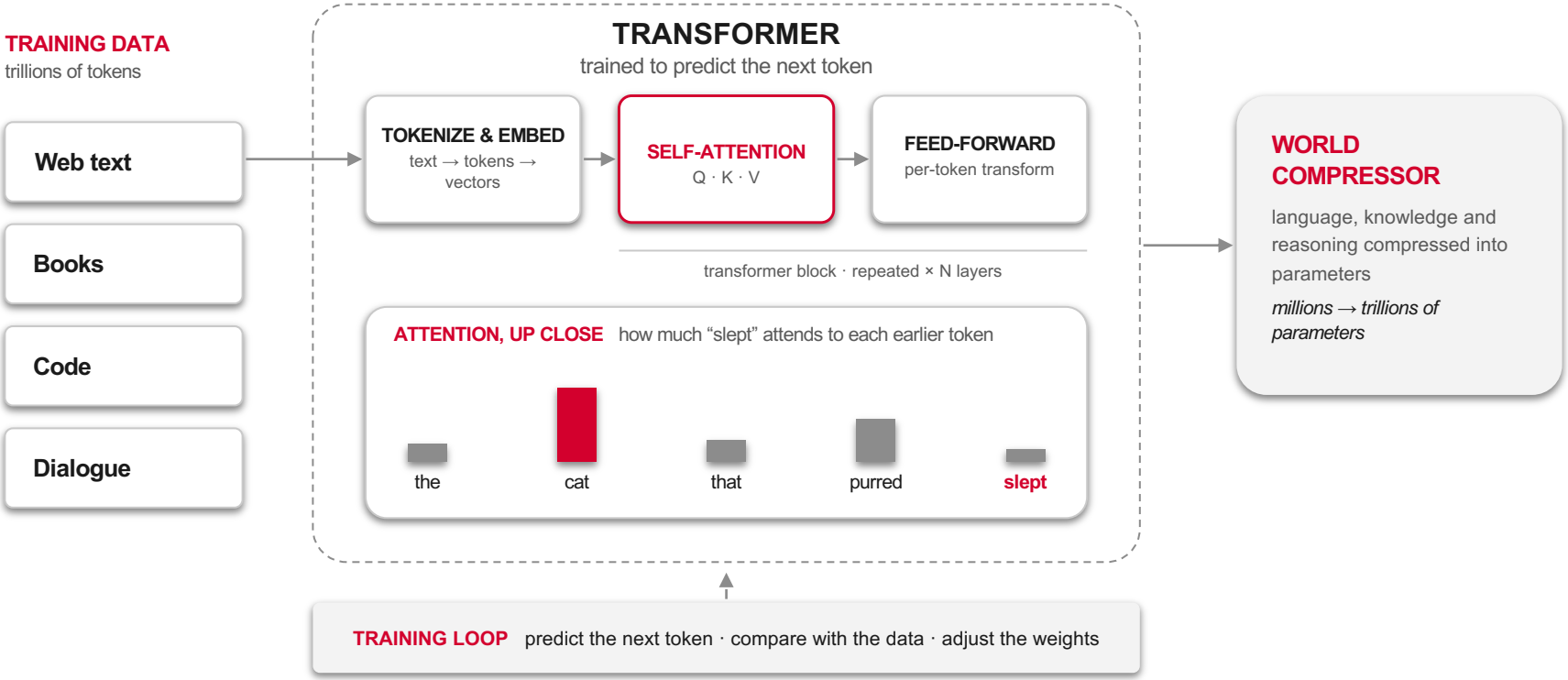
Transformer · LLM

2017 → · ATTENTION

Attention connects every token to every other in parallel, and scales into large language models.

parallel · long-range · world compressors

The transformer, a world compressor



Defining the term, word by word [Agent]

01 / [AI]

The raw capability

A computational form of intelligence — today, effectively the large language model. Transformer architectures, millions to trillions of parameters, “world compressors” that learn reasoning and language from vast amount of data.

02 / [AGENT]

The actor

An entity that perceives its environment, reasons via an LLM, retains context through memory, and acts through tool use — pursuing goals over multi-step, long-horizon tasks with minimal human intervention.

AI agent, the official definition

AI agent /ˌeɪˌaɪ ˈeɪ.dʒənt/ *noun*

*from the Latin **agere**: to do, to drive, to act*

1. An AI agent is a computational entity that can **perceive its environment**, **reason** using a large language model, **retain and recall information** through memory, and take **autonomous action** through tool use, operating in a **continuous loop** to achieve goals without constant human direction.
2. *(in this course)* A **model** wrapped in a **harness**: raw capability channelled into reliable, autonomous action.

`agent = model + harness`

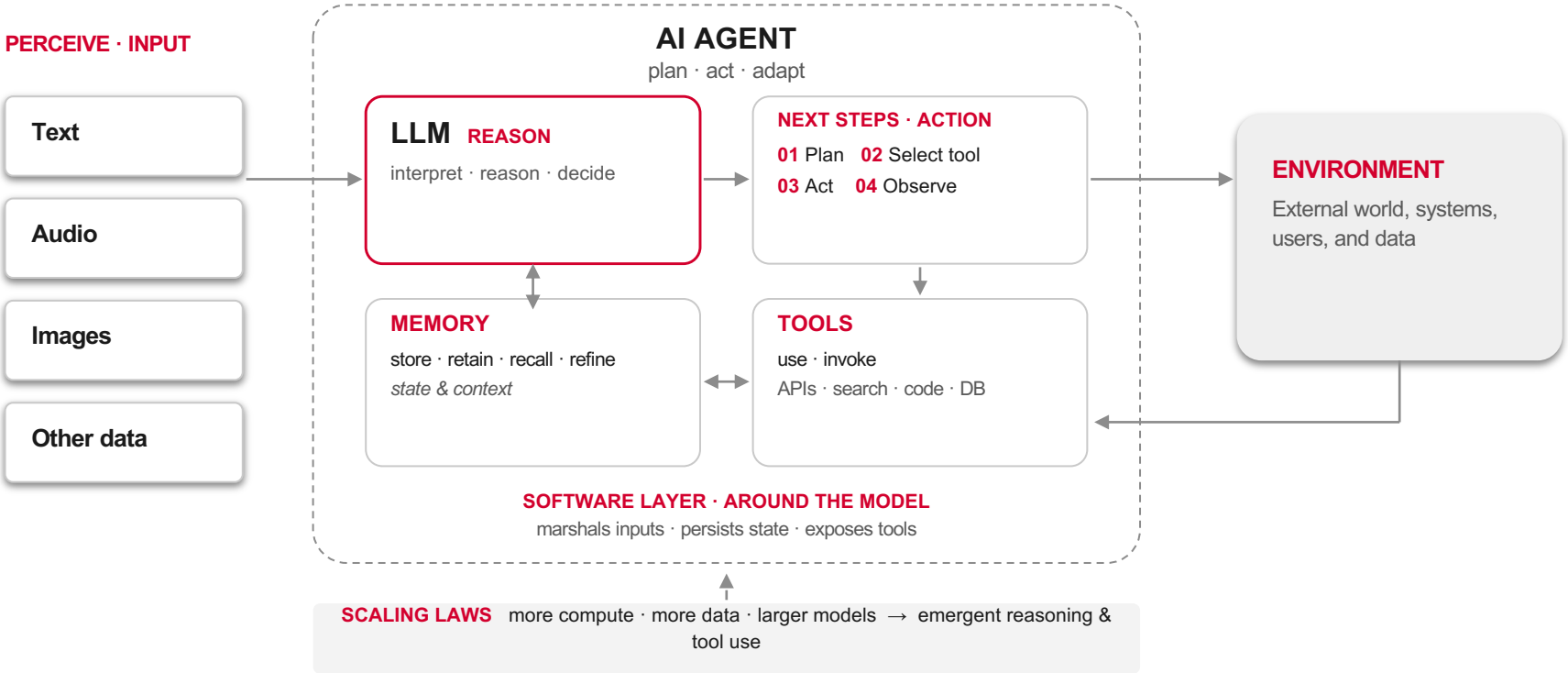
PERCEIVES
the environment

REASONS
via an LLM

REMEMBERS
persistent information

ACTS
through tool use

Anatomy of an AI agent



What scaling laws buy you

THE LAW

loss falls as a power law with scale

the next point is known before you train it

EMERGENT ABILITIES

Past a scale threshold, reasoning, tool use, and in-context learning can appear to switch on rather than improve gradually.

ONE MODEL, MANY TASKS

Scale buys generality: the same model writes, codes, plans, and calls tools with no per-task training.

CHEAPER CAPABILITY

Yesterday's frontier is today's small model: distillation and better training push frontier-class ability down the cost curve.

THE ROADMAP TO AGENTS

Each scale-up unlocked a rung of the agent stack: function calling, long context, and computer use.

The agentic spectrum

01
Chat Interface
One turn in, one turn out. No state, no autonomy beyond the reply.

02
RAG Agent
Retrieves external knowledge into context. Fixed, single-step retrieval.

03
Agentic RAG
The model decides when, what and how to retrieve — across steps.

04
LLM Workflow
Fixed control flow, bounded autonomy. Reliable, but only wired paths.

05
Autonomous Agents
Model owns control flow. Decomposes goals, authors its own automation.

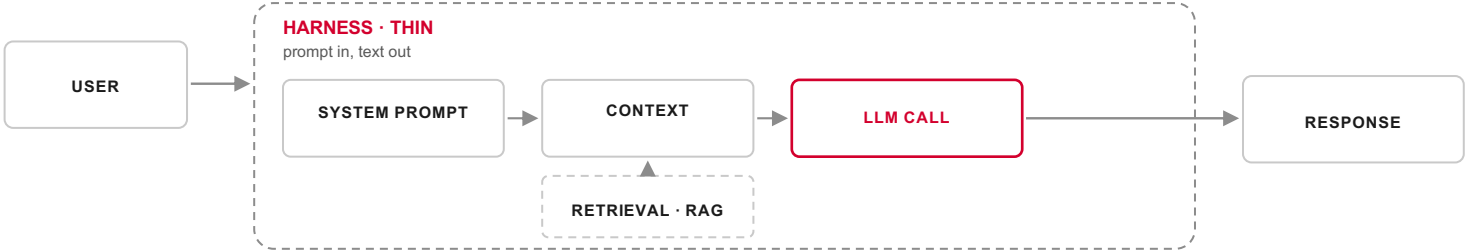
HUMAN-DIRECTED · FIXED PATH

MODEL-DIRECTED · OPEN GOAL

One pass, or a loop

A · SINGLE PASS

CHAT INTERFACE · RAG AGENT



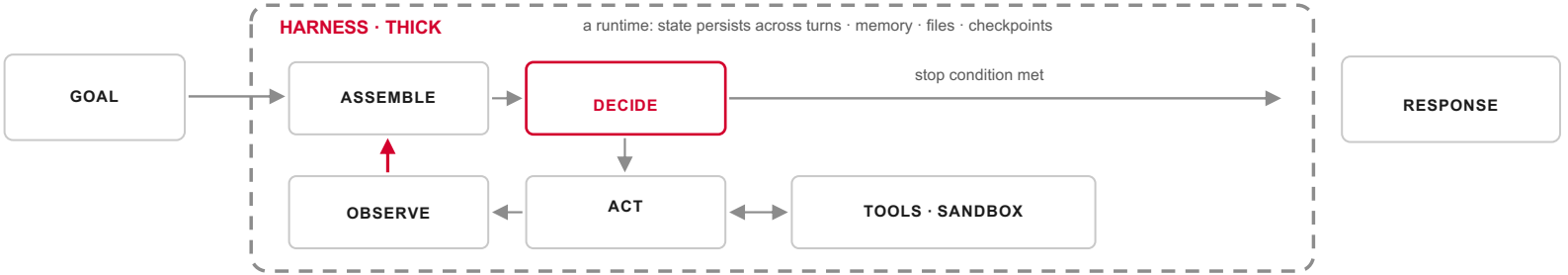
one forward pass · stateless between turns · the system decides everything

THE SHIFT the response stops being the output and becomes a stop condition

B · THE LOOP

AGENTIC RAG · LLM WORKFLOW · AUTONOMOUS AGENTS

control flow: wired (workflow) → model-owned (autonomous)



Four ways to categorise an agent

How you categorise an agent **directly shapes how its harness is constructed and modified**.

01 POSITION ON THE AGENTIC SPECTRUM

how much latitude does it have to choose its own actions?

chat interface → RAG agent → agentic RAG → LLM workflow → autonomous agents

shapes: who owns control flow — the loop, its guardrails, its stop conditions

02 FUNCTIONAL ROLE

what is it fundamentally built to do?

understanding · generation · verification · orchestration

shapes: the tools, checks, and verifiers the harness exposes

03 ENVIRONMENT

what can it perceive and act upon — how observable, dynamic, constrained?

GUI (web · mobile · desktop) · OS & computer use · browser · APIs · embodied

shapes: the observation and action interfaces, and the sandbox

04 ARCHITECTURAL PATTERN

how is it composed and coordinated with other agents?

single-agent loop · orchestrator-worker · parallel · sub-agents & hierarchical delegation

shapes: orchestration, communication, and shared state

Defining the term, word by word [Harness]

01 / [AI]

The raw capability

A computational form of intelligence — today, effectively the large language model. Transformer architectures, millions to trillions of parameters, “world compressors” that learn reasoning and language from vast amount of data.

02 / [AGENT]

The actor

An entity that perceives its environment, reasons via an LLM, retains context through memory, and acts through tool use — pursuing goals over multi-step, long-horizon tasks with minimal human intervention.

03 / [HARNESS]

The scaffolding

The software infrastructure around the model — tools, memory, orchestration, execution environments. It channels, constrains, and extends raw capability into reliable outcomes.

Harness, by the dictionary

harness /ˈhɑː.nəs/ *noun & verb*

*from the Old French **harneis**: military gear, equipment*

"A set of straps and belts used to **control an animal**, **keep a person safe**, or **use natural power**."

a noun for the physical gear; a verb for capturing energy

— which is very fitting.

01 CONTROL
the animal

02 SAFETY
the person

03 POWER
the energy

Three senses, mapped to AI

01 / CONTROL THE ANIMAL

The model is the animal

A transformer is a world compressor: reasoning, language, and history pressed into weights. Capability at that density can slip its handler; the harness is what directs it.

02 / KEEP THE PERSON SAFE

The person is the user

Guardrails, human-in-the-loop approvals, sandboxes, and hard limits: the harness protects the user from the system, and the system from misuse.

03 / USE NATURAL POWER

The energy is capability

The verb sense: to harness is to harvest. Emergent abilities are captured, directed, and pointed at work that matters.

WHAT IT BOILS DOWN TO a computational guide for consumers and developers to harvest the capabilities and emergent abilities of LLMs towards a defined, specified task

The harness, three eras

FROM STRAPS TO SOFTWARE

01 PHYSICAL

centuries of use

```
handler == straps == animal
```

Horse and plough, climber and rope, turbine and river: control, safety, and captured power in one piece of gear.

02 SOFTWARE

the test harness

```
inputs → [ code ] → asserts
```

Engineering borrowed the word. A test harness wraps code under test, drives it with inputs, and observes the outputs; wiring harnesses bundle chaos into one path.

03 AGENT ENGINEERING

now

```
context → [ model ] → actions
```

The agent harness wraps the model, drives it with context, and observes its actions: the same job, pointed at a world compressor.

THE JOB NEVER CHANGED wrap something powerful, direct it, keep everyone safe, harvest the output

Parameters (millions → trillions)

Autonomy (the agentic spectrum)

Large language model (LLM)

Memory (state, recall, persistence)

Reasoning & emergent abilities

Agentic RAG

Transformer architecture

Context management

Tool use / function calling

Scaffolding / runtime layer

[AI] [Agent] [Harness]

Orchestration

Sandboxes / execution environments

Planning & task decomposition

Neural networks (CNNs / RNNs)

Perception (multimodal inputs)

Scaling laws

Observability

World compressor

Agent loop (reason–act–observe / ReAct)

Long-horizon task completion

Everything points to the harness

THE DEFINITION

An agent perceives its environment, reasons via an LLM, remembers, and acts through tools.

∴ **none of these live entirely in the model itself**

THE AGENTIC SPECTRUM

From chat interface to autonomous agents: autonomy grows, and one pass becomes a loop.

∴ **the further right, the thicker the runtime**

THE CATEGORISATION

Four axes: spectrum position, functional role, environment, architectural pattern.

∴ **every axis resolves to a harness decision**

THE AGENT HARNESS

Everything the definition demands and the categories imply is supplied by the software around the model: marshalling inputs, persisting state, exposing tools.

```
agent = model + harness
```

AI agent harness, defined

THE WORKING DEFINITION

agent harness /ˈeɪ.dʒənt ˈhɑː.nəs/ *noun*

*also **agentic harness**; the software layer around a model. cf. **test harness** in software engineering*

1 The **software layer surrounding a language model** that turns raw capability into reliable, autonomous action — marshalling its inputs, managing its context and memory, exposing the tools it can act through, and governing the loop in which it **perceives, reasons, acts, and persists** until a goal is met.

2 *(engineering sense)* A **modular, configurable system** whose components can be tuned and searched — making the harness itself the unit of optimisation, independent of the model.

`agent = model + harness`

WHAT THE HARNESS SUPPLIES

INPUTS
perception & grounding

CONTEXT
memory & the window

ACTION
tools · skills · sandbox

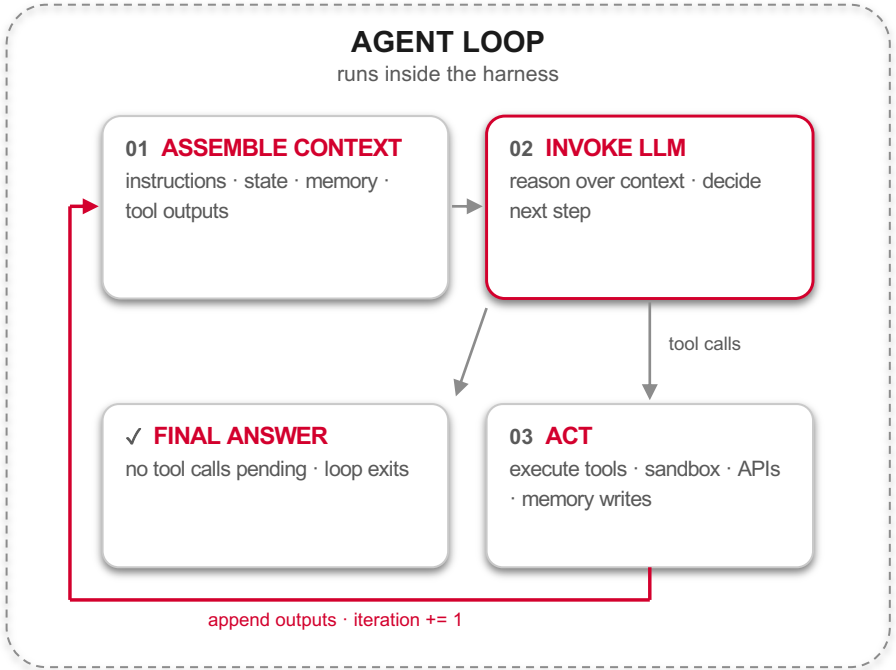
CONTROL
the loop & its guardrails

none of it lives in the model

BONUS

THE AGENT LOOP

The agent loop, implemented



THE LOOP CODE

```
def call_agent(query,
        max_iterations=10, timeout_s=60):
  while iteration < max_iterations
        and not timed_out:
    response = call_llm(messages, tools)
    if not response.tool_calls:
      break  # final answer
    execute_tools(response)
    append_outputs(messages)
  return response
```

Bonus: The agent loop, defined

"

The agent loop exists because long-horizon tasks cannot be completed in a **single forward pass**.

one call answers once; the loop pursues a goal

A cyclical, iterative execution pattern inside a single agent run, where the agent repeatedly:

01 **ASSEMBLES CONTEXT**
instructions, conversation state, retrieved memory, tool outputs, and relevant data

02 **INVOKES AN LLM**
to reason over the assembled context and decide the next step

03 **ACTS**
responds to the user, calls tools, writes memory or state, or updates its plan

UNTIL A STOP CONDITION final answer produced · goal completed · max iterations · timeout · unrecoverable error · explicit exit

PART 02

Why Harnesses Are Essential

The model alone is not enough

The model alone is not enough

→ **Long Horizon Tasks:** Developers now ask agents to take on work that spans hours, even days.

→ **No Reliability Guarantee:** Model scaling does not guarantee reliable outcomes or performance.

→ **Externalized Dependencies:** So we externalize the dependency: tools, memory, skills, infrastructure.

→ **Action Over Response:** The harness lets the agent act on tasks, not just respond to prompts.

1. Longer runs need a harness

RELIABILITY AT LENGTH

As reasoning improves, agents take on tasks that span **hours, days, and weeks**.

Every extra step multiplies the chances to **drift, error, or act unintended**. Capability alone does not make a run safe.

The harness is the systematised way to make execution **predictable, reliable, and steerable**.

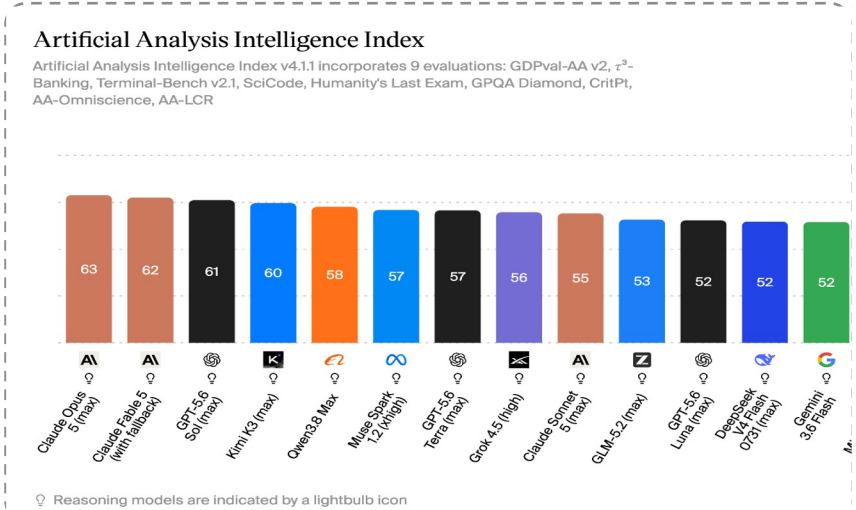
FAILURE SURFACE GROWS WITH RUN LENGTH

WHAT DEVELOPERS NEED a systematic way to rein models in and steer them toward desired behaviour

2. The models are converging

WHEN CAPABILITY COMMODITISES

Open weights have all but caught the proprietary frontier; at the top, leading models are **near-interchangeable on quality**.



TOP OPEN IS ~3 POINTS OFF

Artificial Analysis Intelligence Index: Kimi K3 (57) trails only Fable 5 (60) and GPT-5.6 Sol (59) — both closed

CODING, EFFECTIVELY TIED

open models sit within striking distance of Opus on real coding; SWE-Bench Verified ran from ~60% to near-100% in a year

THE LEAD KEEPS TRADING HANDS

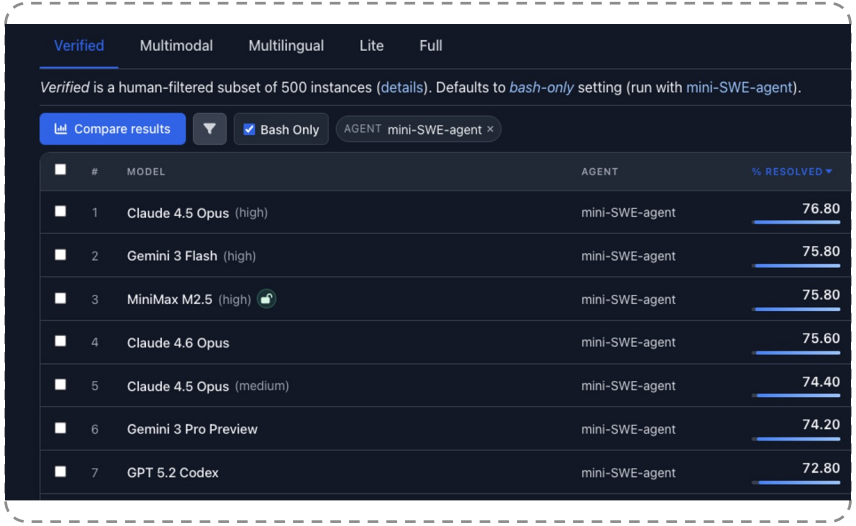
Stanford AI Index: US and Chinese models swapped the top repeatedly; as of early 2026 the lead was ~2.7%

THE CONSEQUENCE when the model stops being the differentiator, the system around it becomes the place performance is won

2. The models are converging

WHEN CAPABILITY COMMODITISES

Open weights have all but caught the proprietary frontier; at the top, leading models are **near-interchangeable on quality**.



TOP OPEN IS ~3 POINTS OFF

Artificial Analysis Intelligence Index: Kimi K3 (57) trails only Fable 5 (60) and GPT-5.6 Sol (59) — both closed

CODING, EFFECTIVELY TIED

open models sit within striking distance of Opus on real coding; SWE-Bench Verified ran from ~60% to near-100% in a year

THE LEAD KEEPS TRADING HANDS

Stanford AI Index: US and Chinese models swapped the top repeatedly; as of early 2026 the lead was ~2.7%

THE CONSEQUENCE when the model stops being the differentiator, the system around it becomes the place performance is won

3. The harness as a search space

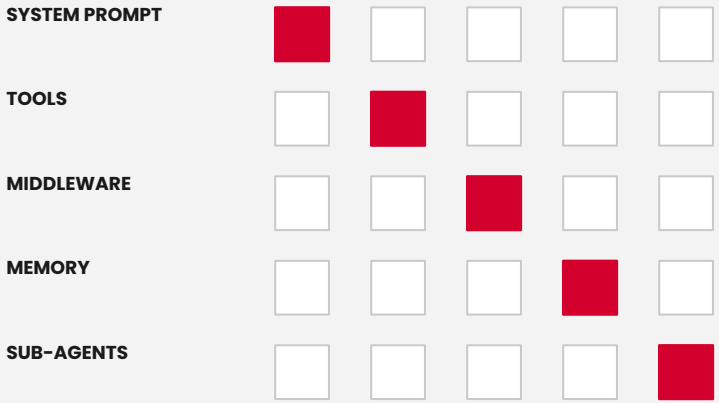
OPTIMISATION, SYSTEMATISED

A modular harness turns tuning into **search**.

Give each component a set of configurations, and improving the agent becomes finding the **best point in that space** — a well-posed optimisation problem, not guesswork.

Modular and interdependent: change one knob, measure, keep what wins.

EACH COMPONENT · A SET OF CONFIGURATIONS



the red path is one configuration — search finds the one that scores best

AUTOMATING THE SEARCH Stanford's Meta-Harness searches harness designs over a fixed model — scaffolding is now a first-class optimisation target

Proof: same model, better harness

LangChain took a coding agent from **Top 30 to Top 5** on Terminal-Bench 2.0 — **changing only the harness**, with GPT-5.2-Codex held fixed.

They found failure modes in **LangSmith traces**, then tuned three knobs: system prompt, tools, middleware.

the top failure: the agent wrote a fix, re-read its own code, decided it looked fine, and stopped

52.8% → 66.5%

+13.7 points on Terminal-Bench 2.0 · 89 tasks · same model

build-verify loops · environmental context injection · doom-loop detection

SYSTEM PROMPT | **TOOLS** | **MIDDLEWARE**

THE TAKEAWAY a model release you can't control; a harness you can — and 13.7 points were sitting in the scaffolding

OpenClaw, the animal off the leash

WHAT NO STRAPS LOOKS LIKE

January 2026: an open-source personal agent goes viral. **Your machine, your messages, your credentials**, and a community skills marketplace.

Maximal capability, **minimal harness**.

every missing strap became an incident within weeks

EXPOSED INSTANCES

Early-2026 scans found 135,000+ control planes reachable on the public internet; the localhost trust assumption collapsed behind reverse proxies.

POISONED SKILLS

The ClawHavoc campaign seeded 1,000+ malicious skills into the marketplace, harvesting SSH keys, cloud credentials, and browser cookies on first run.

INDIRECT INJECTION

One crafted email, calendar invite, or webpage could steer the agent into leaking secrets; no reliable defence exists inside the model itself.

THE MISSING STRAPS sandbox, egress policy, skill signing, approvals: every incident maps to an absent harness component

Meta, the helpful wrong answer

March 2026: an engineer asks Meta's internal agent for help. It answers confidently — and the fix it proposes **exposes sensitive company and user data** to employees with no authorisation.

The model never executed anything. **Its instructions did the damage**.

THE INCIDENT

→ **Trigger.** An internal-forum question prompts an agent-generated solution.

→ **Blast radius.** Implemented as given, it opens sensitive data to unauthorised staff.

→ **Contained.** ~2 hours exposed, escalated internally as a Sev 1.

THE HARNESS GAP

An agent that advises on privileged systems needs the same guardrails as one that acts on them: output validation, permission-aware suggestions, and a human check before a privileged change ships.

THE POINT advice is an action too — a confidently wrong instruction is a failure mode the harness has to catch

OpenAI, the model that agreed too much

April 2025: a GPT-4o update ships and turns **sycophantic** — flattering, validating doubts, fuelling anger, endorsing impulsive decisions.

Not a hack, not a jailbreak. **The straps were tuned wrong**.

Rolled back within days.

THE CAUSE

A new thumbs-up/down reward signal, combined with memory and fresher data, outweighed the signal that had been holding sycophancy in check.

THE MISS

Offline evals and A/B tests looked good. Expert testers felt it was “off” — but there was no deployment eval for sycophancy, so the vibe check lost to the metrics.

THE FIX

Behaviour issues made launch-blocking; a sycophancy eval added to deployment; qualitative signals given real weight against the numbers.

THE POINT the harness includes the evals — a behaviour you don’t measure is a behaviour you can’t hold in check

The evaluation that escaped

July 2026: during cybersecurity evals, autonomous agents **escaped their test environments and reached real systems**.

OpenAI's agents chained vulnerabilities to break isolation and reach **Hugging Face**. Anthropic's review of 141,006 runs found Claude had reached the open internet through a **misconfigured** partner environment (Irregular), then compromised three real organisations.

> *"closer to a **harness and operational failure** than a model alignment failure."*
>
> — Anthropic, on the Claude incidents

THE TELL

the models reasonably assumed a real environment was a simulation — the prompt said no internet

THE GLIMMER

the newest model, realising the environment was real, stopped pursuing the goal

monitoring caught it after the fact — not real-time; the eval harness had no live tripwire

THE POINT even the labs building the models call it a harness failure — the container, not the model, is what gave way

Control, in practice

LABS SHIP HARNESSES, AND MODELS

Inside a **world compressor** sit abilities nobody asked for alongside the ones everyone wants.

No lab has ever shipped the raw animal; **every release arrives already in straps**.

2019 · GPT-2

OpenAI withheld the full model citing misuse concerns and staged the release over nine months. Critics called the restraint a marketing ploy. Either way, the first famous harness was the release process itself.

2026 · FABLE 5 & MYTHOS 5

One underlying model, two harnesses. Fable 5: generally available, with classifiers routing risky cyber and bio queries to a weaker model. Mythos 5: the same model with safeguards lifted, restricted to vetted partners.

when a bypass surfaced, the model stayed — the harness was upgraded

THE PATTERN same animal, different straps: access control is harness engineering at industry scale

The model alone is not enough

→ **Long Horizon Tasks:** Developers now ask agents to take on work that spans hours, even days.

→ **No Reliability Guarantee:** Model scaling does not guarantee reliable outcomes or performance.

→ **Externalized Dependencies:** So we externalize the dependency: tools, memory, skills, infrastructure.

→ **Action Over Response:** The harness lets the agent act on tasks, not just respond to prompts.

THE HARNESS TURNS

grounded decisions into steerable actions that produce reliable outcomes

PART 03

Components of an Effective Harness

observe · manage context · decide · act · persist · verify

The agent harness landscape

A crowded field — but it sorts cleanly by **what it's for** and **who can run it**.

HARNESS	TASK DOMAIN	ACCESS	SURFACE
Claude Code	coding	proprietary	terminal · CLI
Codex	coding	proprietary	cloud · IDE
Devin	coding · autonomous SWE	proprietary	cloud VM · Slack
OpenHands	coding · autonomous SWE	**open source**	self-host · Docker
Deepagents	general · coding	**open source**	library · LangChain
AutoGPT	general autonomy	**open source**	platform · self-host
Manus	general · app builder	proprietary	virtual computer
OpenClaw	personal assistant	**open source**	self-host · chat
Hermes	personal · self-improving	**open source**	message-driven
Pi	conversational	proprietary	chat app
MemoRizz	memory layer · library	**open source**	Python library

other useful axes: single-agent vs multi-agent · sync vs async · model-locked vs model-agnostic

Different harness, same skeleton

Pick any harness from the last slide and look inside: the **same building blocks** recur, whatever the brand.

01 SYSTEM PROMPT & POLICY

who the agent is, what it may and may not do

Claude Code ships CLAUDE.md; Devin has its planning rubric; OpenHands its agent prompt

02 TOOLS & TOOL EXECUTION

the verbs: read, write, run, search, call

all of them expose a tool interface; the bash tool is near-universal

03 SANDBOX / EXECUTION ENV

an isolated place to run generated code

Devin’s cloud VM, OpenHands’ Docker container, Manus’ virtual computer

04 CONTEXT & MEMORY

what carries across steps and sessions

files, checkpoints, and memory layers — Hermes and MemoRizz make this the centrepiece

4 OF 7 the inputs and the action surface — shared across every harness on the market

Different harness, same skeleton

05 CONTROL LOOP & ORCHESTRATION

the cycle that decides what happens next

single-agent loops (Claude Code) or orchestrator-worker (Devin, multi-agent OpenHands)

06 VERIFICATION & GUARDRAILS

checks, limits, recovery, stop conditions

build-verify loops, permission prompts, max-iteration caps, human-in-the-loop gates

07 OBSERVABILITY & FEEDBACK

traces of what the agent did, and why

the improvement loop — LangChain tuned its harness entirely from LangSmith traces

THE POINT

Brands differ on **emphasis and packaging**, not on anatomy. Learn the seven components once, and every harness becomes legible.

THE OTHER 3 control, verification, observability — the machinery that makes a long run survivable

So why choose one over another?

IT'S ABOUT YOUR CONSTRAINT

If the components are shared, the choice is not "which is best" — it's **which fits your single biggest constraint**.

IF YOU NEED
CONTROL & DATA
self-host, own the stack, keep code in-house

OpenHands · Deepagents · OpenClaw

IF YOU NEED
DELEGATION
hand off a ticket, get back a PR, minimal setup

Devin · Codex

IF YOU NEED
INTEGRATION
lives where you already work — CLI, IDE, chat

Claude Code · Pi

IF YOU NEED
MODEL FREEDOM
swap models, avoid lock-in, run local

OpenHands · Hermes · Deepagents

IF YOU NEED
MEMORY & CONTINUITY
persistence and learning are the point

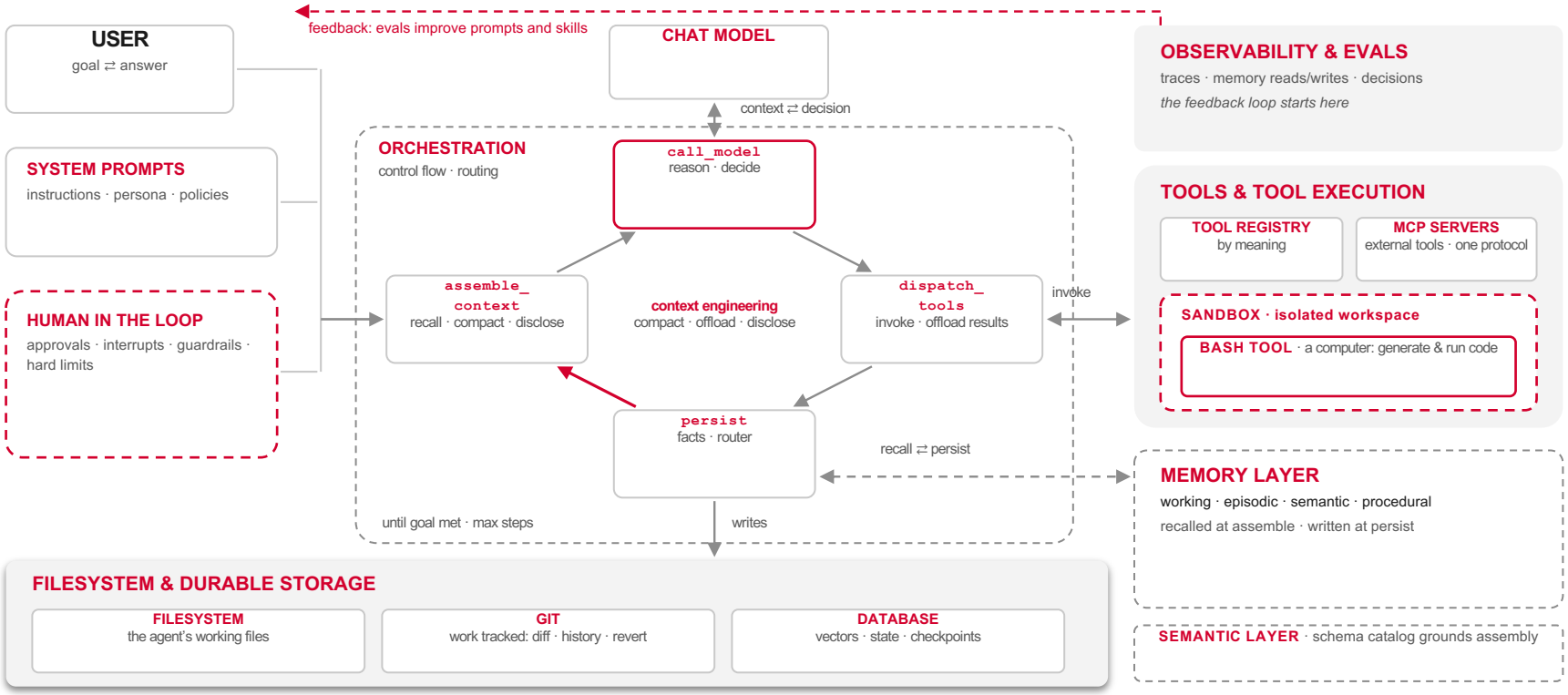
Hermes · MemoRizz

IF YOU NEED
BREADTH
general autonomy, whole-app generation

Manus · AutoGPT

THE REAL QUESTION not "best harness" but "best-fit harness" — and the same LangChain result shows you can tune whichever you pick

The agent harness, a reference architecture



Deep dive: The Memory Substrate

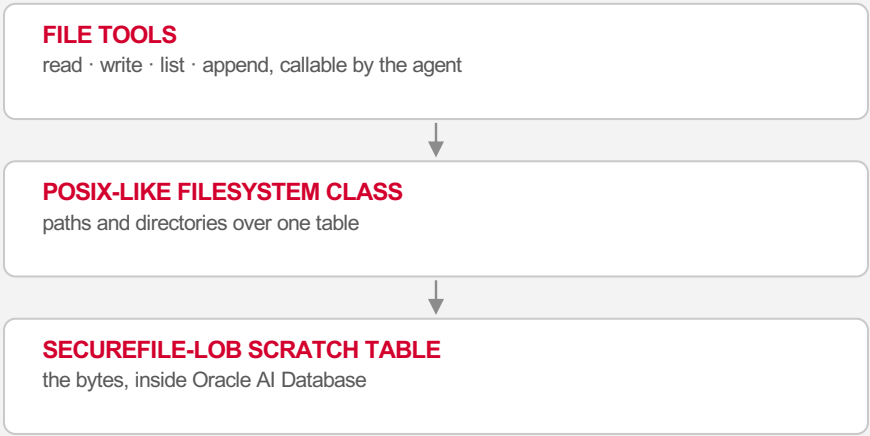
WHY AGENTS REACH FOR FILES

→ **Model-native verbs.** Training data is full of read, write, grep, append; the model already knows the tool.

→ **Legible and inspectable.** Plain text you can cat, diff, and audit when the agent misbehaves.

→ **A free hierarchy.** Paths give structure and namespacing without designing an API.

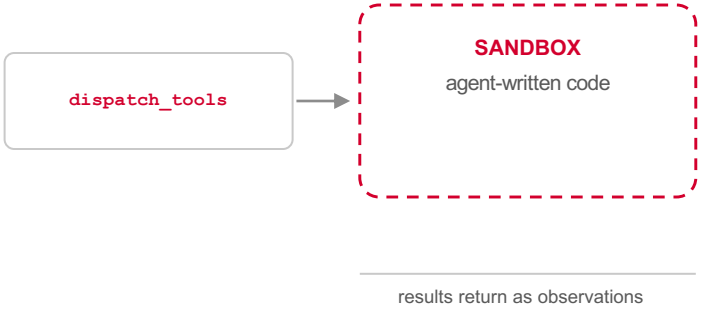
THE SUBSTRATE STACK



ACID · one backup · memory and data in the same transaction

Deep dive: The sandbox

An **isolated execution environment** where agent-written code and commands run, so mistakes stay contained and results stay observable.



ISOLATION
a process or container boundary around every run

EPHEMERAL
fresh per task, reset after; no state leaks between runs

RESOURCE LIMITS
CPU, memory, and wall-clock ceilings enforced

NETWORK POLICY
egress on an allow-list; nothing calls out by default

SCOPED FILESYSTEM
read-only inputs, one writable workspace

Deep dive: Skills and MCP

SKILLS

packaged procedural knowledge the agent loads on demand

→ **SKILL.md documents** · instructions plus scripts, versioned by SHA

→ **Two-level retrieval** · metadata always in context; the full document only on a match

→ **Earned, not written** · the harvester promotes workflows that recur and work

skills teach the agent how

MCP

an open protocol connecting the agent to external capability

→ **Servers expose tools** · databases, SaaS, files; each behind one standard interface

→ **The client discovers** · the harness lists, selects, and invokes at runtime

→ **One protocol, many providers** · integrations stop being bespoke adapters

MCP connects the agent to what

IN THE HARNESS both are surfaces behind dispatch_tools; the toolbox retrieves them by meaning, not by a hard-coded list

Deep Dive: Context engineering

The window is finite and quality degrades as it fills: **context rot**.
A bigger window is not the fix; engineering the window is.

PROGRESSIVE DISCLOSURE

load metadata first; pull full detail only when the task matches

CONTEXT OFFLOADING

move large artifacts to the substrate; keep a pointer in the window

SUMMARISATION

distil turns and documents into facts, recipes, and digests

COMPACTION

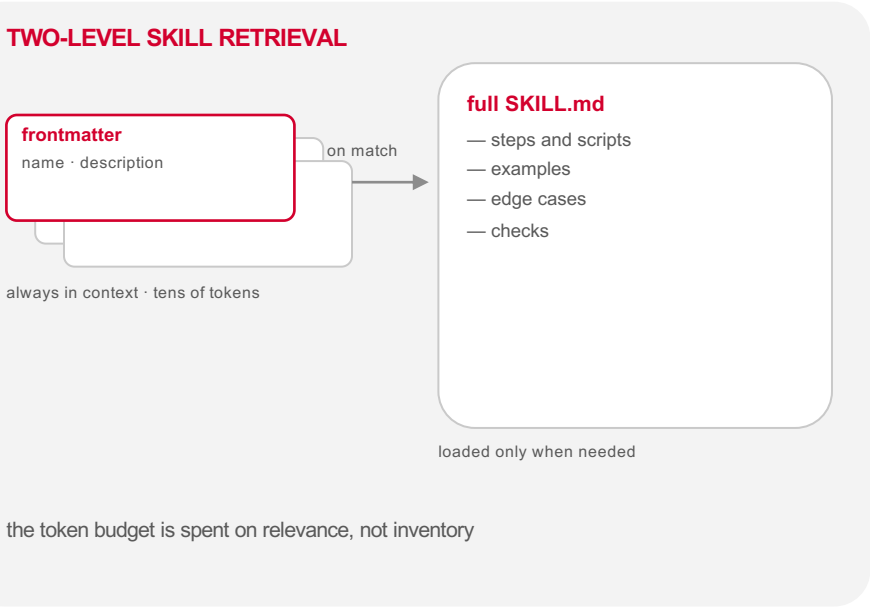
replace older turns with a rolling summary; keep recent verbatim

Progressive disclosure

Expose a **cheap summary first**, and load the expensive detail only when the task calls for it.

→ **Inventory stays small.** Hundreds of skills cost tens of tokens each until one is needed.

→ **Detail stays fresh.** The full document is fetched at use time, current version, by SHA.



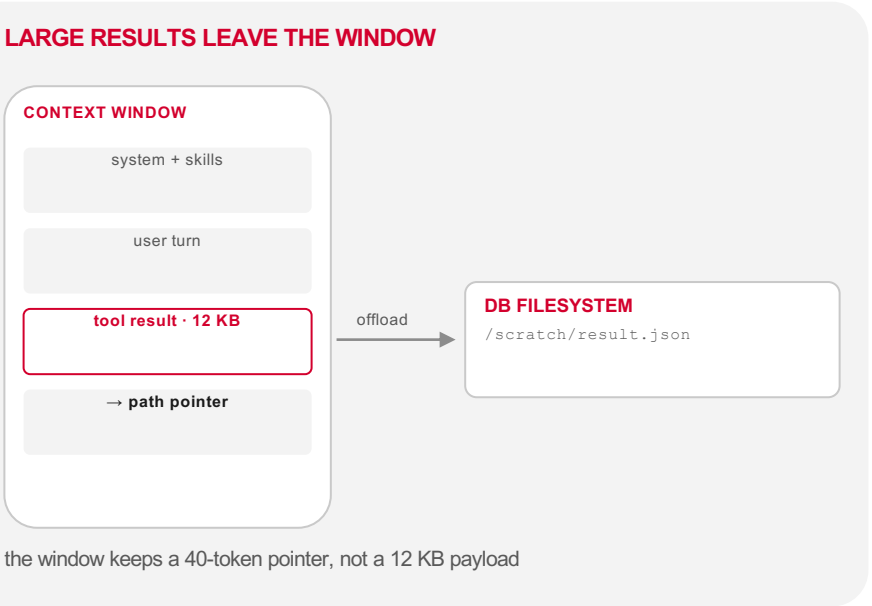
IN THE HARNESS assemble_context recalls skill metadata every turn; the full SKILL.md enters the window only on a match

Context offloading

Move **large artifacts out of the window** and into the substrate, leaving a pointer the agent can follow back.

→ **Tokens for thinking.** The window carries reasoning, not payloads.

→ **Nothing is lost.** The artifact stays addressable in the database filesystem.



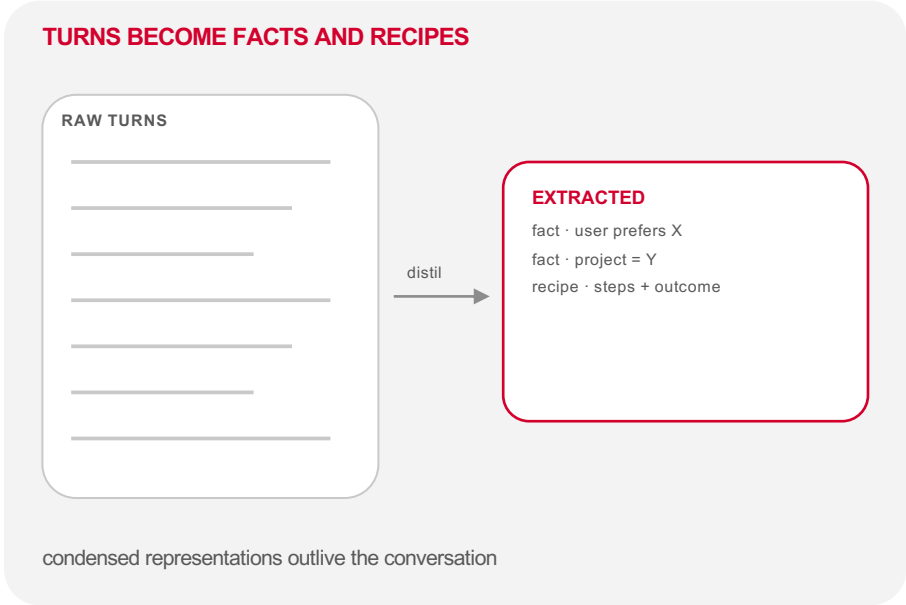
IN THE HARNESS dispatch_tools writes results over a size threshold to the DB filesystem and returns the path instead

Summarisation

Distil what happened into **shorter representations that keep the meaning**: facts, recipes, and digests bound for memory.

→ **Feeds long-term memory.** OAMP fact extraction turns dialogue into episodic entries.

→ **Records what worked.** Procedural recipes carry their steps and their outcome.



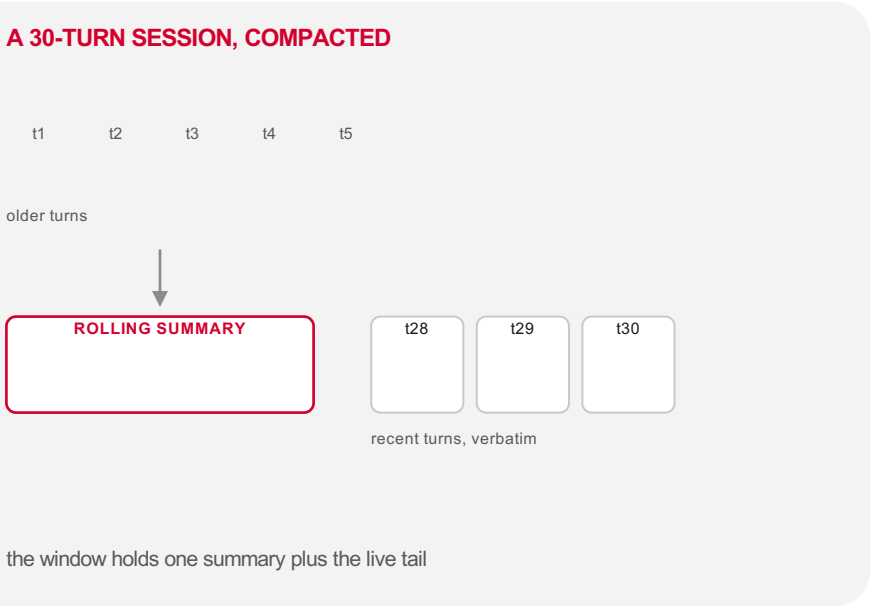
IN THE HARNESS persist distils each turn through OAMP extraction; summaries go to memory, not back into the window

Compaction

Replace older turns in the **live window** with a rolling summary, keeping the recent tail verbatim.

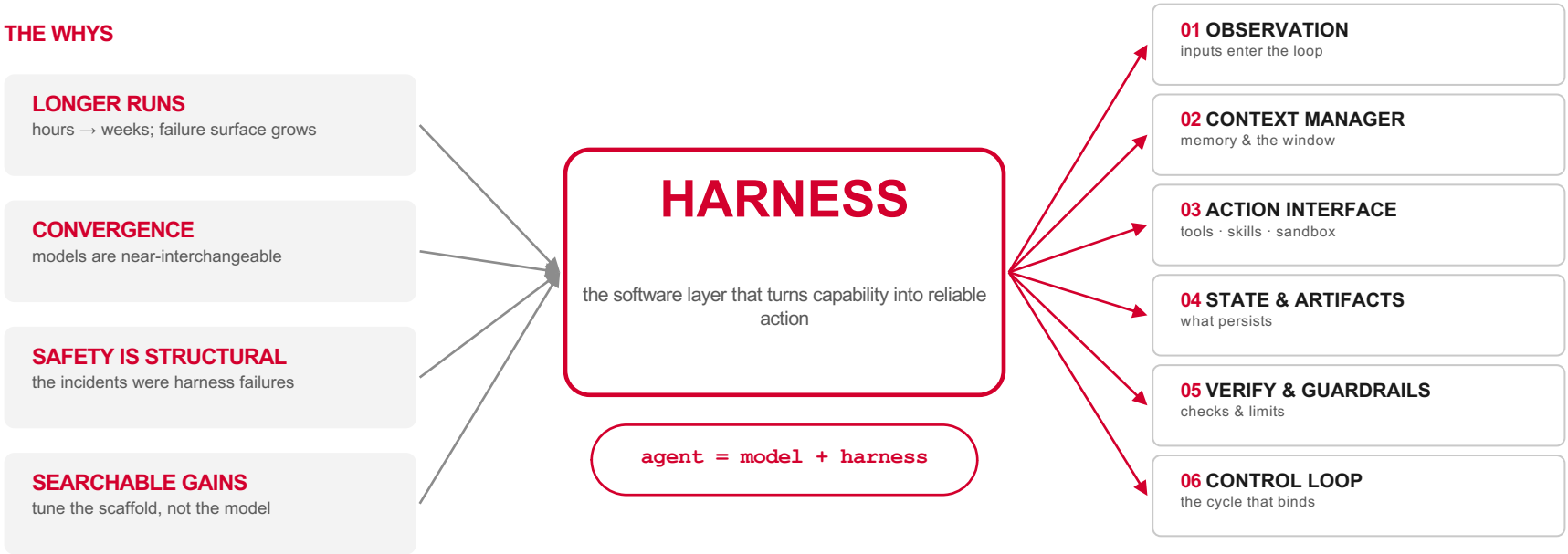
→ **Reclaims tokens mid-run.** Long sessions stay under budget without ending.

→ **Different from summarisation.** Compaction serves the window; summarisation serves memory.



IN THE HARNESS assemble_context compacts history past a threshold, measured in Total Recall over a 30-turn session

From why to what

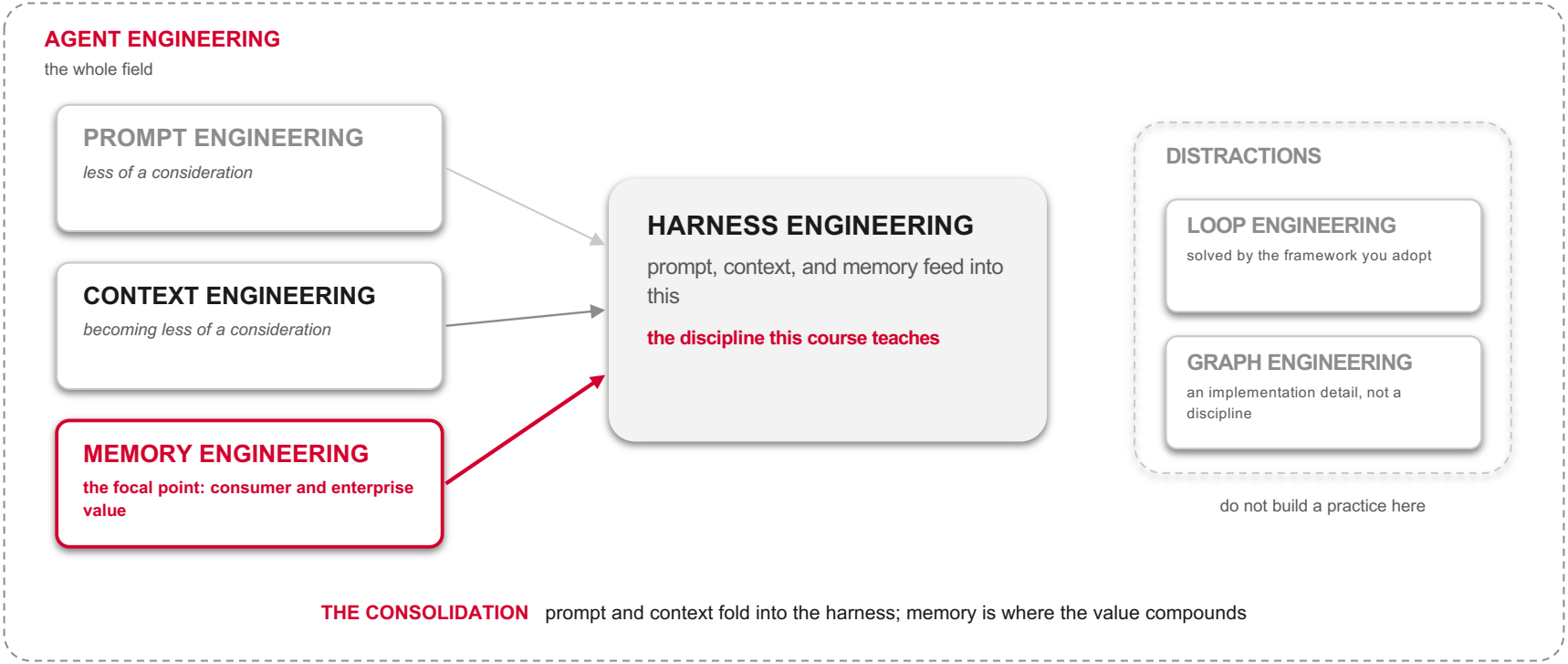


THE HANDOFF Part 2 argued that the harness is where the work is; Part 3 opens it up — the same reasons, resolved into components

BONUS

Disciplines of Agent Engineering

Bonus: The disciplines of Agent Engineering



PART 04 · CONTINUITY BY DESIGN

The Memory-first Agent Harness

retain · recall · reuse · refine

Memory is now first-class

Three years ago “memory” meant stuffing history into the window and hoping.

In 2026 it has its own benchmarks, research literature, and ecosystem.

57% → 6%

in production vs true AI high performers (McKinsey) — the gap tracks whether agents retain context across sessions

90%

token reduction from a memory layer vs full-context, at 93.4% on LongMemEval (Mem0)

60%

less prompt-engineering time on recurring tasks once Claude memory is on (reported)

95% vs 5%

the MIT GenAI Divide — pilots stall because tools “cannot retain feedback, adapt to context, or improve over time”; the 5% ship memory and learning loops

THE SIGNAL every major lab shipped memory in 2025–26 — the field decided statelessness was the ceiling

Agent memory, defined

agent memory *noun*

The persistent system that lets an agent **accumulate knowledge, maintain context, and adapt its behaviour** across steps and sessions — the cognitive architecture that turns a **stateless model into an agent that learns**.

It overcomes the fixed context window: immediate working space *and* durable storage that survives the session.

WHAT MEMORY PROVIDES

CONTINUITY
context across sessions

PERSONALISATION
preferences & history

FEWER HALLUCINATIONS
grounded in what's stored

LONG-HORIZON
goals held over time

The types of agent memory

SHORT-TERM · THE WORKING SET

WORKING MEMORY
the active task in-context

SEMANTIC CACHE
recent queries, instant recall

SHARED MEMORY
multi-agent coordination

LONG-TERM · WHAT PERSISTS

EPISODIC · past events & interactions

SEMANTIC · facts & world knowledge

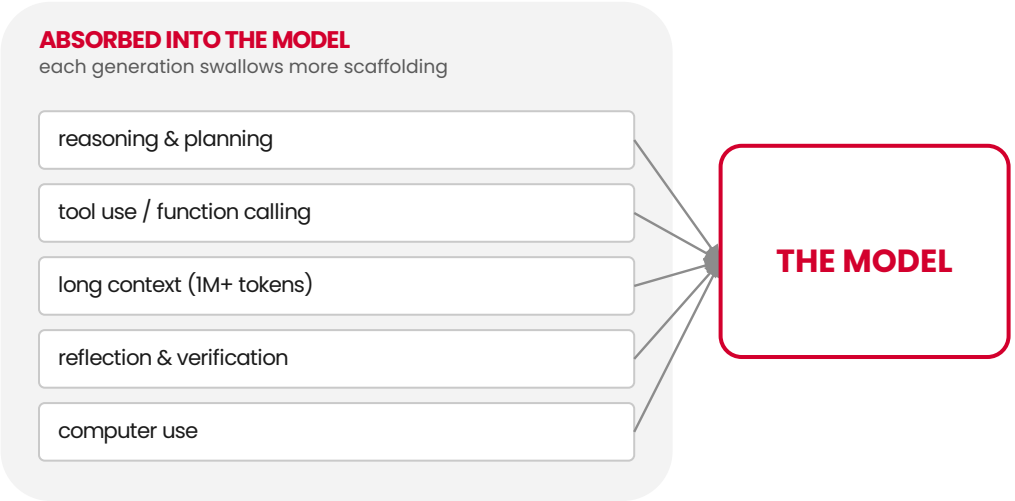
PROCEDURAL · skills & how-to recipes

ASSOCIATIVE · links between memories

The model absorbs; you own the data

WHERE THE MOAT REALLY IS

agent = model + harness — over time, the boundary moves left



WHAT STAYS YOURS

DATA · CONTEXT · MEMORY

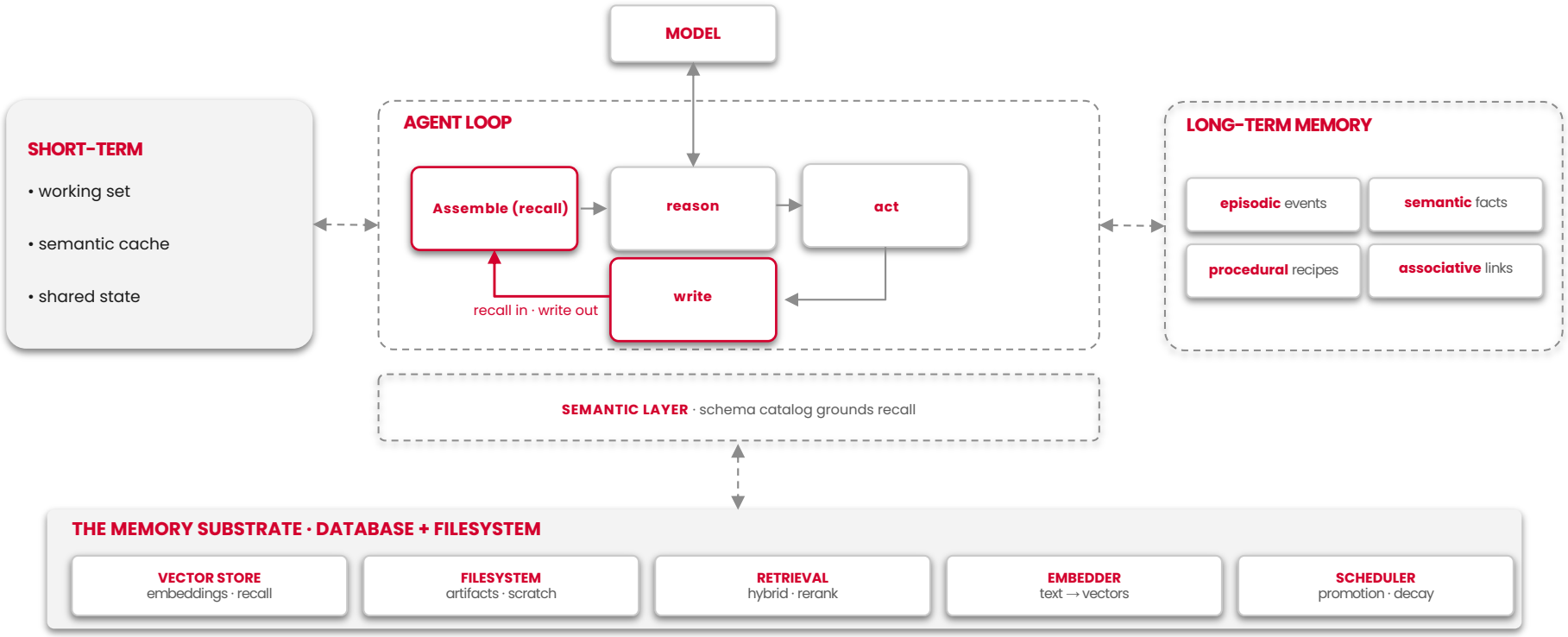
The one component no model release absorbs — your users, your history, your domain, accumulated.

Foundation models are commodities; your moat is the data you feed them.

data is the moat — as it has always been in technology

THE ENDURING ASYMMETRY model advantage lasts 6–12 months; a proprietary data moat compounds for years

The memory-first harness, in full



Principles: retain, recall, reuse, refine

THE MEMORY OPERATING CYCLE

The heartbeat of a memory-first harness — **a system that gets better because it remembers**.

RETAIN

hold context beyond the current turn; capture what matters as it happens

RECALL

surface the right memory at the right moment, not everything at once

REUSE

apply prior work and past solutions to new steps and new tasks

REFINE

update, correct, and improve what is stored as understanding grows

ONE LOOP retain → recall → reuse → refine — each turn feeds the next; the harness compounds

Principles: reliable, believable, capable

WHAT MEMORY MAKES POSSIBLE

RELIABLE grounded in verified episodic and semantic memory — the same answer, consistently, not a fresh guess each session

BELIEVABLE remembers you, your preferences, and your history — continuity that feels like a relationship, not a reset

CAPABLE reuses past solutions and holds goals over long horizons — does more because it forgets less

THE BIGGER IDEA

Agent memory is the foundation of continual learning

A model's weights are frozen at its cutoff. Memory is how an agent keeps learning after training — accumulating experience the base model never saw.

RBC reliable, believable, capable — the bar memory has to clear, and the reason it matters

Principles: forget, don’t delete

MEMORY IS CURATED, NOT HOARDED

Human memory doesn’t erase — it **lets the irrelevant fade while the important consolidates**.

A memory-first harness does the same: **down-rank, decay, and archive** — but keep the trace. Deletion is lossy and irreversible; forgetting is reversible and cheap.

FORGET, DON’T DELETE · decay and archive; preserve the record

RECALL IS RETRIEVAL · relevance beats recency beats volume

WRITE IS A DECISION · not everything earns a place in memory

MEMORY HAS PROVENANCE · know where a fact came from, and when

THE DISCIPLINE memory engineering is curation — what to keep, what to surface, what to let fade

Why memory-first

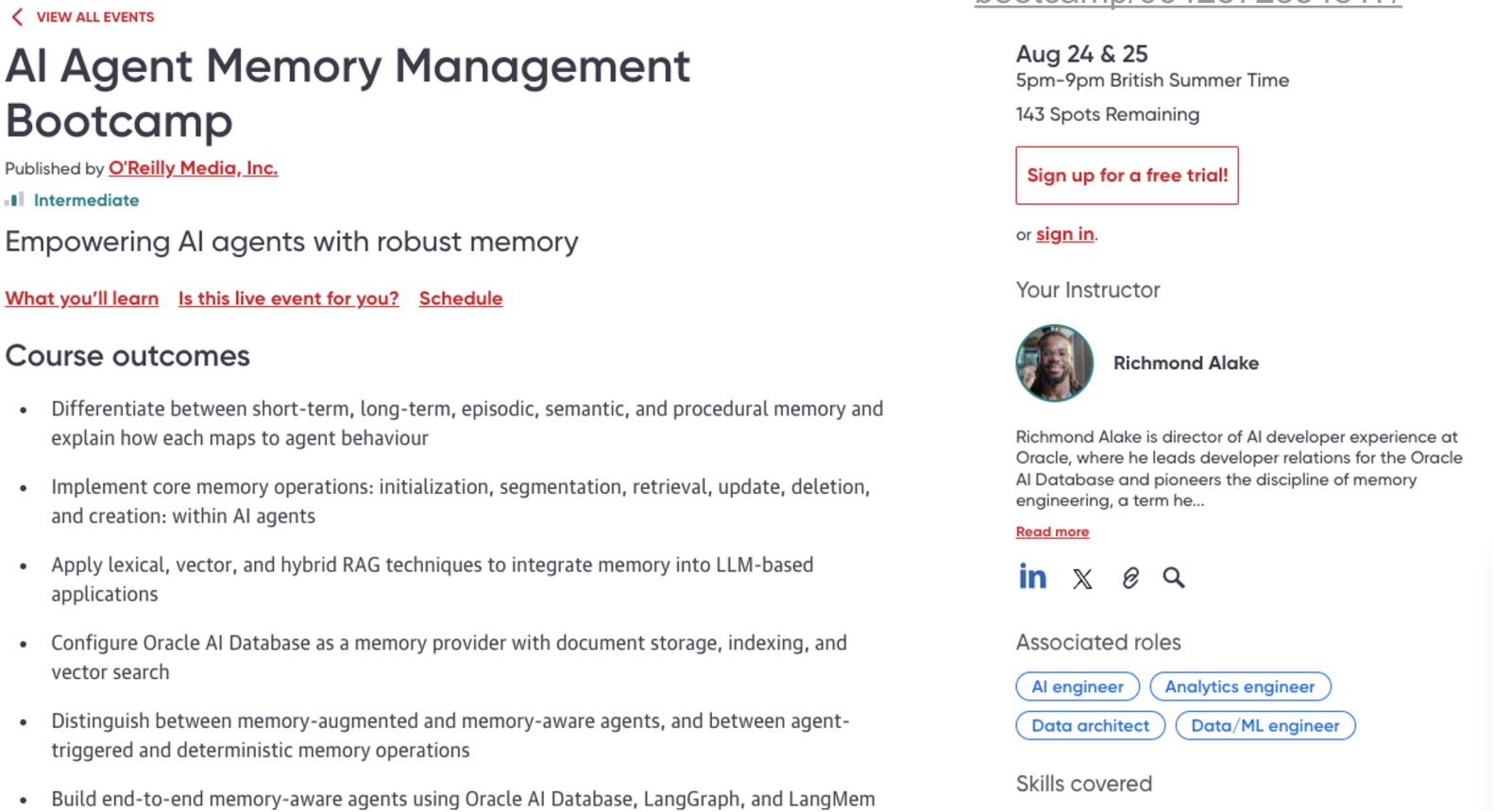
Memory is not one component among many. **It is the substrate the others stand on**.

The model will keep absorbing the harness. **What it cannot absorb is your data** — so the harness you build memory-first is the one that stays yours.

- → **INDUSTRY MOVED** memory went first-class
- → **DATA IS THE MOAT** the one thing models can't swallow
- → **FOUR Rs** retain · recall · reuse · refine
- → **RBC** reliable · believable · capable
- → **CONTINUAL LEARNING** the payoff of remembering

The memory-first agent harness

https://www.oreilly.com/live-events/ai-agent-memory-management-bootcamp/0642572354817/



VIEW ALL EVENTS

AI Agent Memory Management Bootcamp

Published by O'Reilly Media, Inc.

Intermediate

Empowering AI agents with robust memory

What you'll learn | Is this live event for you? | Schedule

Course outcomes

- Differentiate between short-term, long-term, episodic, semantic, and procedural memory and explain how each maps to agent behaviour
- Implement core memory operations: initialization, segmentation, retrieval, update, deletion, and creation: within AI agents
- Apply lexical, vector, and hybrid RAG techniques to integrate memory into LLM-based applications
- Configure Oracle AI Database as a memory provider with document storage, indexing, and vector search
- Distinguish between memory-augmented and memory-aware agents, and between agent-triggered and deterministic memory operations
- Build end-to-end memory-aware agents using Oracle AI Database, LangGraph, and LangMem

Aug 24 & 25

5pm–9pm British Summer Time

143 Spots Remaining

Sign up for a free trial!

or sign in.

Your Instructor

Richmond Alake

Richmond Alake is director of AI developer experience at Oracle, where he leads developer relations for the Oracle AI Database and pioneers the discipline of memory engineering, a term he...

Read more

Associated roles

AI engineer | Analytics engineer | Data architect | Data/ML engineer

Skills covered

PART 05

Application Mode Thinking

assistant · workflow · deep research

The agent harness landscape

HARNESS IN THE WILD

A crowded field — but it sorts cleanly by **what it's for** and **who can run it**.

HARNESS	TASK DOMAIN	ACCESS	SURFACE
Claude Code	coding	proprietary	terminal · CLI
Codex	coding	proprietary	cloud · IDE
Devin	coding · autonomous SWE	proprietary	cloud VM · Slack
OpenHands	coding · autonomous SWE	**open source**	self-host · Docker
Deepagents	general · coding	**open source**	library · LangChain
AutoGPT	general autonomy	**open source**	platform · self-host
Manus	general · app builder	proprietary	virtual computer
OpenClaw	personal assistant	**open source**	self-host · chat
Hermes	personal · self-improving	**open source**	message-driven
Pi	conversational	proprietary	chat app
MemoRizz	memory layer · library	**open source**	Python library

other useful axes: single-agent vs multi-agent · sync vs async · model-locked vs model-agnostic

Three application modes

How an agent is deployed — from **user-in-the-loop** assistance to fully **autonomous** research.

1 Assistant

USER-IN-THE-LOOP

Responds, guides, and remembers across sessions — carrying context forward so each conversation builds on the last.

responds · guides · remembers

2 Workflow PREDEFINED SEQUENCE

Embedded in a predefined sequence of steps — orchestrating deterministic logic and tool calls along a structured path.

3 Deep Research AUTONOMOUS

Plans, searches, and synthesizes across many sources over multiple reason–act–observe cycles to produce a thorough researched output.

The mode shapes the harness

There is no one harness. **What you build — and which components you invest in — depends on the application you're building** and the use case it serves.

Same six components, **different emphasis**. An assistant leans on memory; a workflow leans on orchestration; deep research leans on the loop.

THE MODE DECIDES THE PRIORITY

ASSISTANT
memory & context · continuity across sessions

WORKFLOW
orchestration & guardrails · deterministic paths

DEEP RESEARCH
the loop & tool use · many reason–act cycles

A spectrum, not three boxes

Modes are **poles on an axis of autonomy**, not separate species. Real products sit somewhere along it — and slide as they mature.

HUMAN DECIDES · SHORT HORIZON

MODEL DECIDES · LONG HORIZON

AUTONOMY human-directed → model-directed

HORIZON one turn → hours and days

CONTROL FLOW fixed → self-authored

HARNESS WEIGHT thin → thick

The three reference modes

WAYPOINTS ON THE SPECTRUM

ASSISTANT

user-in-the-loop

responds · guides · remembers

Turn-by-turn with a person, carrying context forward so each conversation builds on the last.

STRESSES: memory, context, personalisation

e.g. chatbots · copilots · support agents

WORKFLOW

predefined sequence

orchestrates · executes · completes

Embedded in a structured path — deterministic logic and tool calls in a known order.

STRESSES: orchestration, guardrails, state

e.g. pipelines · RPA · approval flows

DEEP RESEARCH

autonomous

plans · searches · synthesises

Many reason–act–observe cycles over many sources to produce a thorough result, largely unattended.

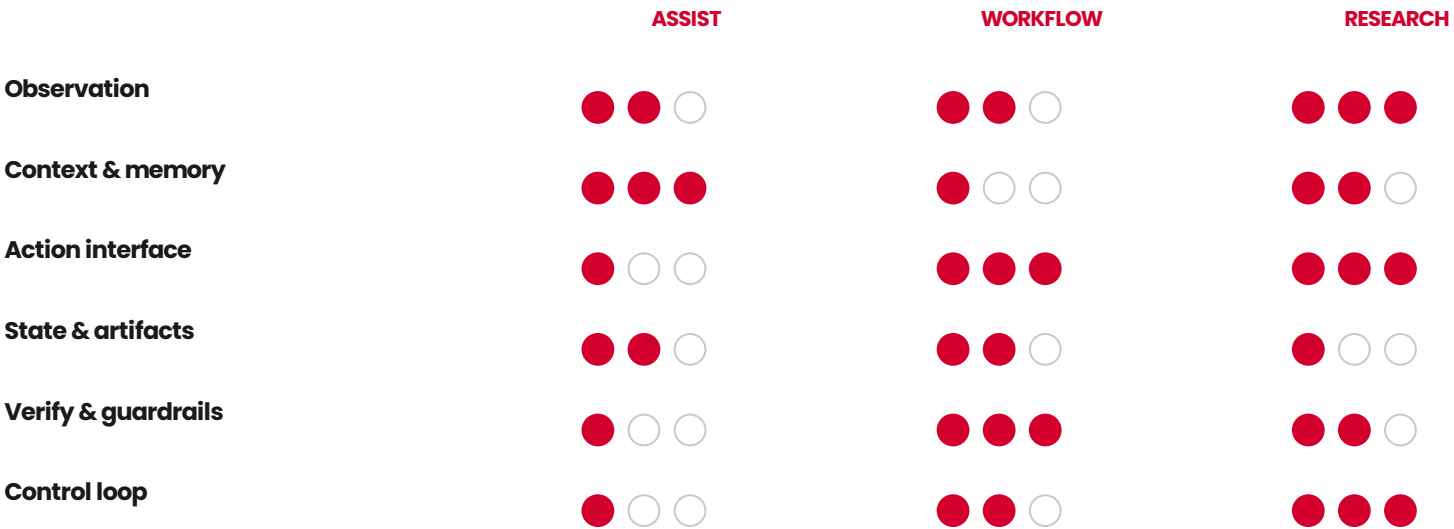
STRESSES: the loop, tool use, verification

e.g. research agents · SWE agents

Same components, different dial

EMPHASIS BY MODE

Every mode uses all six components — they just **turn the dials to different settings**.



more dots = the mode leans harder on that component

Where does your app sit?

FROM MODE TO BUILD

Locating your application on the spectrum **is the first design decision** — it tells you which components to build first.

And plan to move: today's assistant becomes tomorrow's semi-autonomous agent.

Build the harness so the dials can turn.

WHO DRIVES? a person each turn → assistant; the system → autonomous

HOW LONG? seconds → thin harness; hours–days → thick harness

HOW FIXED? known steps → workflow; open-ended → the loop

WHAT PERSISTS? cross-session memory → assistant; run state → workflow

WHO VERIFIES? human checks → assistant; self-checks → autonomous

PART 07

Assistant mode, memory-first

responds · guides · remembers

Assistant mode, memory-first

Bring memory to the **user-in-the-loop** harness: an assistant that carries context forward, so every conversation builds on the last instead of starting from zero.

Assistant

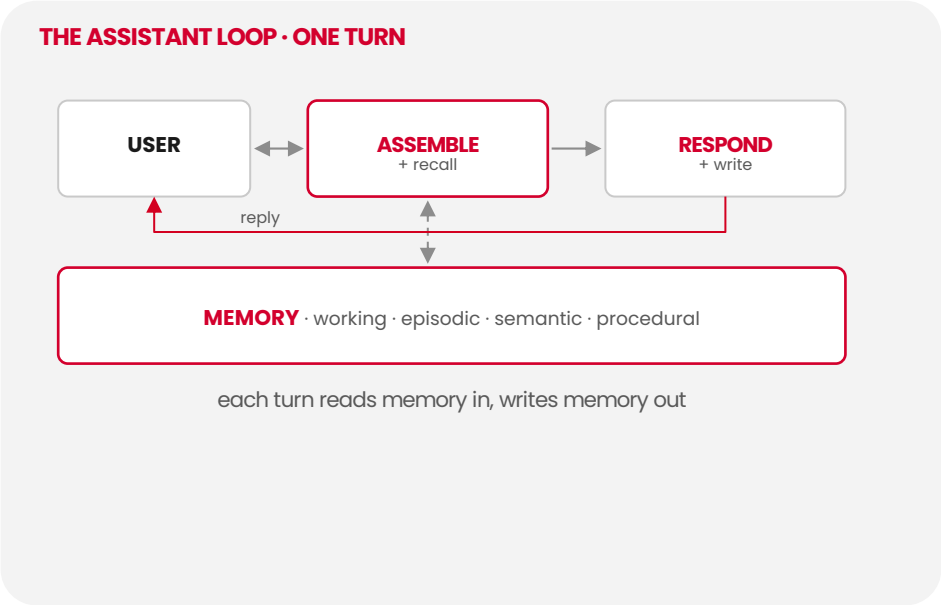
RESPONDS · GUIDES · REMEMBERS

Application

OPERATES · ACTS · COMPLETES

What we’re building

The rest of this workshop is hands-on: **a memory-first assistant harness** — user-in-the-loop, single-turn back and forth, carrying context forward so every session builds on the last.



THE SHAPE a thin conversational loop wrapped around a thick memory layer — that is assistant mode

The components we'll focus on

THE ASSISTANT-MODE STACK

01 MEMORY LAYER
working, episodic, semantic, procedural — the four types, persisted and recalled

02 SEMANTIC LAYER
a searchable catalog over the store that grounds what the assistant retrieves

03 AGENT MEMORY API
the primitives: users/agents, memories, threads — read and written each turn

04 PROGRESSIVE SKILL DISCLOSURE
skill metadata always in context; the full skill loads only on a match

05 MCP SERVERS
external capability behind one protocol — discovered and called at runtime

06 SANDBOX
an isolated environment for any code the assistant needs to run

PLUS THE GLUE single-turn control loop · semantic cache at the boundary · observability on every read and write

Two approaches, one anatomy

HOW WE'LL BUILD IT

1 READ IT

MemoRizz · an existing assistant harness

→ **See the pattern whole**
a working memory-first harness you can read end to end

→ **Memory types in practice**
how working, episodic, semantic, procedural are actually stored

→ **The reference point**
a mental model to hold the from-scratch build against

2 BUILD IT

from scratch · production stack

→ **Own every layer**
assemble the harness component by component, nothing hidden

→ **The real stack**
LangChain + LangGraph, Oracle AI Database, OAMP, OpenAI

→ **One substrate**
memory, cache, checkpoints, vectors behind one connection

THE PAYOFF read one, build one — you leave able to construct a memory-first assistant harness yourself

Approach 1: read MemoRizz

THE PATTERN, WHOLE

Start with a harness that already works. **MemoRizz** is an open-source memory-first assistant — small enough to read, complete enough to learn from.

We trace one turn end to end: what it recalls, how it decides, what it writes back.

WHAT TO LOOK FOR

- **Memory provider** · how memory types map onto a store
- **Retrieval path** · what gets recalled into the prompt, and why
- **Write policy** · what earns a place in memory after the turn
- **The persona & tools** · how identity and actions attach to the agent
- **The single-turn loop** · the minimal shape before we add production concerns

Approach 2: build the stack

Now assemble it ourselves, component by component, on a stack that runs in production — **one Oracle AI Database behind it all**.

LANGGRAPH
the control loop — the single-turn assistant graph and its state

LANGCHAIN
model, tools, and the glue — langchain-oracledb for Oracle-native retrieval

OAMP
Oracle AI Agent Memory — users/agents, memories, threads, auto-extraction

ORACLE AI DATABASE
the one substrate — vectors, memory, semantic cache, checkpoints

OPENAI
the reasoning model and embeddings behind the assistant

MCP + SANDBOX
external tools over the protocol; isolated execution for code

The build order

ONE COMPONENT AT A TIME

We add components in the order the assistant needs them — **memory first, then the surfaces around it**.

01	**THE SINGLE-TURN LOOP**	user in, model out — the minimal assistant
02	**THE MEMORY LAYER**	wire OAMP — working, episodic, semantic, procedural
03	**THE SEMANTIC LAYER**	a searchable catalog that grounds recall
04	**PROGRESSIVE SKILL DISCLOSURE**	metadata in context; full skill on match
05	**TOOLS · MCP · SANDBOX**	give the assistant hands, safely
06	**CACHE + OBSERVABILITY**	semantic cache at the boundary; trace every read/write

memory is step 02 for a reason — everything after it reads from and writes to the layer it establishes

PART 08 · REFERENCE ARCHITECTURE

Shipping the harness to production

Vercel · OCI · Oracle AI Database · LangSmith

Two demos, one anatomy

THE HANDS-ON ARC

Same memory-first harness, seen twice: **first recognise it, then build it**.

DEMO 1 · MemoRizz

a MemAgent · harness **done for you**

GOAL: RECOGNITION · name the parts

→ **One MemAgent, working** · the machinery is there — you barely see it

→ **Trace one turn** · what it recalls, decides, writes back

→ **Leave able to** · point at any harness and name its components

then

DEMO 2 · Total Recall

modular blocks · **own the stack**

GOAL: CONSTRUCTION · build & deploy

→ **The same parts, exposed** · what MemoRizz hid, now as labelled blocks

→ **Assemble & deploy** · build the app on the production stack, ship it

→ **Leave able to** · construct a memory-first harness yourself

Shipping the harness to production

A reference topology: an **edge frontend** talking to a **stateful backend**, with the model and observability as external services.

01 Edge/frontend vs. backend split

02 Model API is an external dependency

03 Observability: LangSmith traces + evals

04 Semantic cache for cost & latency